

Стать IT-шником может каждый



Введение в pandas. Тип данных Series



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Введение

Основной блок

Задание для закрепления

Вопросы по основному блоку

Заключение



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Повторение

- Скорость NumPy
- Последовательное и строчное расположение
- Работа с библиотекой `numpy.random`
- Немного про распределения



Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Введение

Библиотека pandas

Тип данных Series

Индексация

Продвинутые операции с Series



Стать IT-шником может каждый



ОСНОВНОЙ БЛОК



Стать IT-шником может каждый



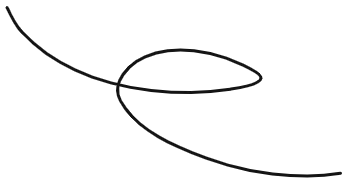
Библиотека pandas





Pandas —

это мощная библиотека для анализа и манипуляции данными, написанная на языке программирования Python. Она предоставляет удобные и быстрые инструменты для работы с табличными данными: таблицы баз данных SQL, Excel-файлы и CSV-файлы.





Установка библиотеки pandas

```
pip install pandas
```



Подключение библиотеки pandas

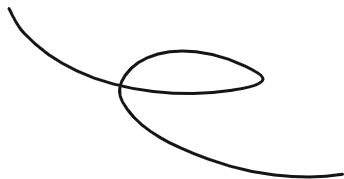
```
import pandas as pd
```

Полезно знать



Series —

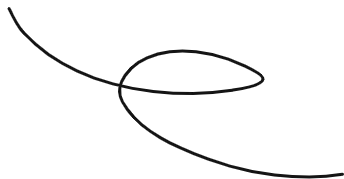
это одномерный массив данных.





DataFrame —

это двумерная структура данных, похожая на таблицу в базе данных или на лист в Excel, которая состоит из строк и столбцов.



Библиотека pandas



- Широко используется в научных исследованиях, финансовом анализе, инженерии, статистике, машинном обучении и многих других областях, где необходима работа с данными.
- Позволяет аналитикам и разработчикам эффективно обрабатывать большие объемы данных, проводить комплексный анализ, строить отчеты и препроцессинг данных для машинного обучения.

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



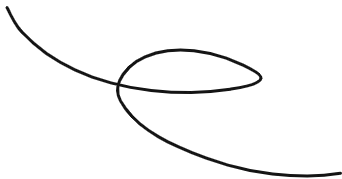
Тип данных Series





Series —

это одномерный массив данных, который может хранить данные любого типа. Элементы в Series индексируются, по умолчанию используется числовой индекс от 0 до N-1, где N – количество элементов в Series. Pandas позволяет задать и использовать именованные индексы.





Особенности Series

- Series поддерживает автоматическое индексирование и возможность задавать пользовательские индексы.
- Пользовательские индексы могут быть числами, строками или даже датами.
- Каждому элементу в Series можно присвоить метку (через индекс).
- Множество встроенных методов для анализа данных: статистические функции, операции сортировки и ранжирования, методы для обработки пропущенных значений.
- Series - строительный блок для DataFrame в pandas.
- DataFrame - контейнер для Series.

Пример создания Series

```
import numpy as np
import pandas as pd

l = [1, 3, 5, np.nan, 6, 8]
s = pd.Series(l)
s
0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
dtype: float64
```

Пример создания Series

```
s.values, type(s.values)
# (array([ 1.,  3.,  5., nan,  6.,  8.]), numpy.ndarray)
s.to_dict()
# {0: 1.0, 1: 3.0, 2: 5.0, 3: nan, 4: 6.0, 5: 8.0}
s.index
# RangeIndex(start=0, stop=6, step=1)
```

Пример создания Series

```
s.describe()
```

```
count    5.000000  
mean     4.600000  
std      2.701851  
min      1.000000  
25%      3.000000  
50%      5.000000  
75%      6.000000  
max      8.000000  
dtype: float64
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Индексация



Индексация

```
s = pd.Series([1, 3, 5, np.nan, 6, 8])
```

```
s[2]
```

```
s[2] = 7
```

```
s
```

```
s[2:5]
```

```
s1 = s[1:]
```

```
s1
```

```
s2 = s[:-1]
```

```
s2
```

Индексация

```
s1 + s2
```

```
0 NaN
```

```
1 6.0
```

```
2 14.0
```

```
3 NaN
```

```
4 12.0
```

```
5 NaN
```

```
dtype: float64
```

Индексация

```
np.exp(s1)  
type(np.exp(s)), type(s)
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Что выведет следующий код?

```
cube = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  
print(cube[1, 0, 1])  
print(cube[:, 1, 0])  
print(cube[:, 1])
```

Индексация по умолчанию

S.index

RangeIndex(start=0, stop=6, step=1)

type(np.exp(s)), type(s)

Наборы данных с индексом, заданным СПИСКОМ

```
i = list('abcdef') # i == ['a', 'b', 'c', 'd', 'e', 'f']
s = pd.Series(l, index=i)
s
```

a	1.0
b	3.0
c	5.0
d	NaN
e	6.0
f	8.0

dtype: float64

Индексация

S['c']

5.0

s[5]

8.0

s.c

5.0

Создание набора данных из словаря

```
s = pd.Series({'a':1, 'b':2, 'c':0})  
s
```

```
a    1  
b    2  
c    0  
dtype: int64
```

Последовательность дат как индекс

```
d = pd.date_range('20160101', periods=6)
d
# DatetimeIndex(['2016-01-01', '2016-01-02', '2016-01-03', '2016-01-04',
#               '2016-01-05', '2016-01-06'],
#               dtype='datetime64[ns]', freq='D')
```

```
s = pd.Series(l, index=d)
```

```
s
```

```
2016-01-01    1.0
2016-01-02    3.0
2016-01-03    5.0
2016-01-04    NaN
2016-01-05    6.0
2016-01-06    8.0
Freq: D, dtype: float64
```

```
s[s > 3]
```

```
2016-01-03    5.0
2016-01-05    6.0
2016-01-06    8.0
dtype: float64
```


Стать IT-шником может каждый



ВОПРОСЫ



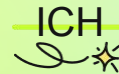
Стать IT-шником может каждый



Продвинутые операции с Series

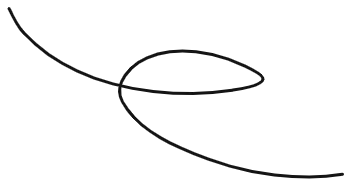


Полезно знать



diff —

это метод, который позволяет посчитать разности соседних элементов.



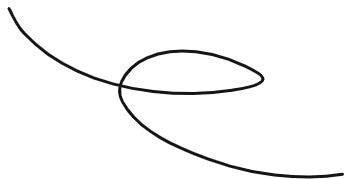
Полезно знать



ICH
e *

periods —

это **ключевое слово**, который отвечает за то, с каким шагом будут считаться разности.

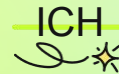


Метод diff

```
s.diff()
```

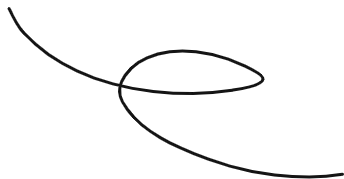
```
2016-01-01    NaN  
2016-01-02    2.0  
2016-01-03    2.0  
2016-01-04    NaN  
2016-01-05    NaN  
2016-01-06    2.0  
Freq: D, dtype: float64
```

Полезно знать



dropna —

это метод, который позволяет избавиться от пропущенных значений.



Метод dropna

```
s.diff().dropna()
```

```
2016-01-02    2.0  
2016-01-03    2.0  
2016-01-06    2.0  
dtype: float64
```


Кумулятивные минимумы

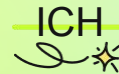
```
s.cummin()
```

```
2016-01-01  1.0  
2016-01-02  1.0  
2016-01-03  1.0  
2016-01-04  NaN  
2016-01-05  1.0  
2016-01-06  1.0  
Freq: D, dtype: float64
```

Кумулятивные суммы

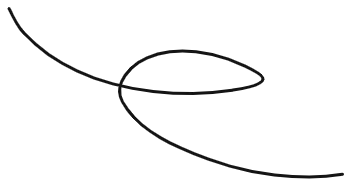
```
s.cumsum()
```

Полезно знать



expanding —

это конструкция, которая позволяет считать произвольные функции кумулятивным способом.



Конструкция expanding

```
s.expanding().apply(np.median)
```

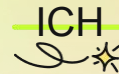
Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Что будет выведено в следующих случаях и почему?

```
s = pd.Series([1, 2, 3, 4, 5, 6])  
t = s[2:]+s[:2]  
print(t)  
print(t.dropna())
```


Стать IT-шником может каждый



ПРАКТИЧЕСКАЯ РАБОТА



Практическое задание

Посчитайте кумулятивное среднее квадратов разностей соседних элементов набора s .
От них можно избавиться при помощи метода dropna.

Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ



Стать IT-шником может каждый



Полезные ссылки

- [pandas.Series — pandas 2.2.1 documentation](#)

Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

